

Student Research Plan Template

PROJECT INFORMATION

Student Name(s): Dakshith Rajkumar

Student Grade: 8

Student School: Fowler Middle School

Title of Project: Project Vel: A Coordinate-Blind, Decentralized Robotic Swarm Protocol for Autonomous Search and Rescue in Disaster Situations

Category ([select from this list](#)) only list the main header, not bulleted subtopics):

Systems Software (SOFT)

Team Project (yes or no): No

Date you plan to start (if research plan is approved): 09/27/2026

ALWAYS USE COMPLETE SENTENCES AND ANSWER THE FOLLOWING

Research Location: *(Where do you plan to do your research check all that apply)*

- ☒ Home
- ☐ School
- ☐ Regulated Research Institution (i.e. University Lab etc.)
- ☐ Place of Business, Commercial Property or Industrial Setting
- ☐ Field or Other Public Location (i.e. City Park, Lake, River, Shopping Mall etc.)
- ☐ Other (Neighbor's house, mentor's house etc.)

Location Address: *(For each location above, list the name and give the street address if available, or if not give location description or GPS coordinates)*

9300 Coit Rd Apt 2013, Plano, TX 75025

Rationale: *(Why is this project important? Include a brief summary of the background that supports your research problem and explain why this research is important scientifically and if applicable, explain any societal impact of your research.)*

When rescue teams send robots into collapsed buildings, mines, or rubble fields, current multi-robot systems break down too easily. Most swarms depend on a central server and strong Wi-Fi or GPS signals to share data and coordinate movements. In a disaster, concrete and metal walls block these wireless signals. If the central coordinator loses connection to the robots, the entire swarm stops working.

Other basic search methods have major drawbacks. Pre-planned routes fail as soon as falling debris blocks a path, while random search patterns waste precious time repeatedly covering areas that were already checked. To fix this, we need a lightweight, decentralized swarm system where robots search efficiently using only local sensor data—without needing GPS, central servers, or constant Wi-Fi connections.

Research Question, Hypothesis/Engineering Goal: *(State your RESEARCH QUESTION(S), HYPOTHESIS/ENGINEERING GOAL(S), EXPECTED OUTCOMES. How is this related to your rationale for choosing this project? Traditional projects will have a hypothesis. Engineering projects will have an engineering goal.)*

Research Question:

Can a decentralized swarm protocol using local sensor data and gradient-based repulsion search complex, GPS-denied disaster spaces efficiently without robots getting stuck on walls or drifting off course?

Engineering Goal:

Design, build, and test a decentralized swarm coverage algorithm (Vel) using Map-Aware Gradient Repulsion and Kinetic Vector Logic to improve search speed and victim discovery in 2D and 3D disaster simulations compared to standard algorithms like Rigid CPP, PSO, ACO, and Levy Flight.

Hypotheses:

- Coverage Efficiency: If we use the Vel protocol with Map-Aware Gradient Repulsion, then the swarm will cover over 95% of the space in under 50 steps, matching rigid CPP efficiency while beating stochastic Levy Flight by over 30%.
- Physical Survivability: If the robots dynamically steer around obstacles using local repulsion forces, then Vel will lower simulated obstacle collisions by over 40% compared to rigid CPP models in collapsed structures.
- Dimensional Scaling: If we turn off inward boundary-restoring forces in 3D environments, then total coverage efficiency will increase by preventing central traffic jams among aerial agents.

Materials: *(List the materials you will be using, including chemicals and amounts and concentrations. Include where you purchased any bacteria or other potentially hazardous biological agents. If you used E.*

coli k-12, make sure to list that as the material. Be extremely detailed with this list. Include the amounts of each material, too. If you are unsure, make your best attempt. This list can always be updated once you get to the final presentation board.)

- Personal computer and Chromebook running Python 3.x, Jupyter Notebook environment, and VS Code.
- Scientific computing and statistical libraries: numpy, pandas, scipy, matplotlib, and seaborn.
- Custom multi-agent simulation framework modeling discrete 2D occupancy grids of size 100 by 100 and 3D voxel arrays of size 40 by 40 by 40.
- Physical Hardware Testbed Units (3 custom LAFVIN 2WD micro-rovers):
 1. 3 x ESP32 dual-core microcontrollers.
 2. 4 x VL53L0X / VL53L1X Time-of-Flight laser distance sensors.
 3. 3 x 2WD LAFVIN robot kits.
 4. 4 x 18650 battery power systems and voltage regulators.
 5. USB programming cables and CP210x / CH340 serial interface drivers.
 6. Jumper Wires
 7. Inline electrical fuses

Procedure: *(This is the section that will cause most students to have their project returned by the teacher for revisions. Describe in detail all procedures and experimental design including methods for data collection, and when applicable, the source of data used. When writing out the steps, please list them out numerically. Do not write this as a paragraph. **Describe only your project. Do not include work done by your mentor or others.** If you will use published surveys, questionnaires, or tests, describe how you obtained these, including required permission if applicable. This should be super detailed. Meaning, that an outside person, when reading this list, could perform the experiment if the supplies were available.)*

Phase 1: Physical Hardware Baseline Validation (25 Percent Experimental Scope)

1. Hardware Assembly and Sensor Calibration: Construct 3 custom 2WD micro-rovers equipped with ESP32 microcontrollers, motor drivers, and Time-of-Flight laser sensors. Calibrate laser range measurement offsets and wheel encoders to maintain uniform movement baselines.
2. Physical Peer-to-Peer Latency and Speed Benchmarking: Flash physical units with point-to-point ESP-NOW communication code. Run 50 physical trial iterations in a bounded test arena to measure peer-to-peer packet transmission latency (recording baseline metrics near 1.1 milliseconds) and real-world linear agent velocity. Record real-world hardware efficiency baselines to ground the physical parameters of the 3D simulation model.

Phase 2: High-Scale 3D Disaster Simulation Framework and Monte Carlo Sweeps (75 Percent Experimental Scope)

3. 3D Environment and Voxel Grid Initialization: Build a custom Python simulation engine to instantiate 3D Shared Spatial Memory voxel lattices (sized 40 by 40 by 40) where individual

voxels store binary status values (zero for unexplored, one for explored). Implement procedural generation algorithms to model diverse 3D real-world disaster topologies, including collapsed multi-story structures, irregular subterranean cave networks, dense structural rubble fields, and narrow industrial ventilation shafts.

4. Environmental Condition Modeling: Program realistic physical constraints into the 3D simulation environment, including signal-shielding concrete walls creating total RF blindspots, stochastic sensor noise, dynamic structural collapses during search execution, and varied obstacle blockage densities.
5. Algorithmic Vector Logic Engine Formulation: Program the Map-Aware Gradient Repulsion algorithm adapted for 3D volumetric space, combining Phase A local void attraction (scanning 3D Moore voxel neighborhood radius $r = 2$) and Phase B global radial expansion upon 100 percent local voxel saturation. Integrate Kinetic Vector Logic using continuous 3D momentum updates (momentum retention $\mu = 0.99$, gradient gain $G = 1.5$, vector gain $\alpha = 0.12$) and sub-step numerical interpolation ($dt = 0.1$) across continuous agent trajectories to eliminate spatial voxel tunneling.
6. Comparative Algorithm Modeling: Implement baseline control algorithms adapted for 3D search spaces within identical Python environments, including 3D Rigid Deterministic Coverage Path Planning and 3D Stochastic Levy Flight.
7. Parameterized Monte Carlo Simulation Sweeps: Execute automated simulation sweeps on personal computer hardware across key independent variables: Algorithm variants: Vel (MAGR/KVL) versus 3D Rigid CPP versus 3D Levy Flight. Disaster topology types: Collapsed Multi-Story Buildings, Subterranean Cave Systems, Dense Structural Rubble Fields, and Confined Industrial Shafts. Real-world environmental conditions: Varying obstacle density (10 percent to 50 percent blockage), active RF signal blindspots, sensor error margins, and active agent hardware failure rates. Swarm scaling: Swarm sizes ranging from baseline groups up to $N = 300$ and $N = 500$ active autonomous agents. Execution limits: Timestep bounds fixed at $T = 50$ and $T = 120$ control steps.
8. Telemetry Logging and Data Capture: Program automatic per-step data logging to capture terminal 3D spatial coverage percentage, victim localization rates, structural collision frequencies, per-step algorithmic execution latency in milliseconds, and continuous positional trajectory paths across all physical and simulated trial runs.

Risk and Safety: *(Identify any potential risks and safety precautions needed. Every experiment has risk and safety concerns, even very minor and unlikely possibilities. This should not be left blank.)*

Physical Rover Collision Risks: Physical testing presents risks of rovers crashing into arena walls or other units, resulting in sharp broken chassis acrylic or loose electrical wiring. All physical testing will be conducted within a controlled, enclosed arena lined with soft foam barriers to absorb physical impacts. Operators will wear safety glasses and work gloves when handling rovers to protect against sharp plastic edges or broken components.

Electrical and Battery Failures: The use of 18650 lithium-ion batteries presents risks of over-current, short circuits, or thermal failure. To mitigate these risks, each rover circuit is equipped with an inline electrical fuse and a dedicated battery protection circuit module. Batteries will be inspected before every run for physical casing damage, kept in enclosed battery holders, and recharged under adult supervision using commercial smart chargers.

Data Analysis: (Describe how you will analyze or compare your data to determine your results. (Will you use tables, graphs or formulas?))

Logged physical telemetry (ESP-NOW latency, physical collision events, real-world step speed) and 3D simulation dataset sweeps will be compiled and analyzed using Python libraries pandas, scipy, matplotlib, and seaborn. Performance across algorithms will be evaluated using descriptive statistics, including mean volumetric coverage percentage, standard deviation, and median time to 95 percent area completion. Comparative plots will be generated, including spatio-temporal convergence curves, collision distribution box plots, and comparative 3D voxel trajectory maps. Statistical significance across treatment groups, disaster topologies, and environmental conditions will be verified using multi-factor ANOVA testing to evaluate overall algorithm performance and dimensional scaling behavior.

Bibliography:

Type in your Bibliography: (Use <https://www.mybib.com/> to properly cite your sources. Why do you need this section? You should be researching to get the background necessary to develop these ideas and plans. List as many sources as needed.)

- [1] H. Choset, "Coverage for Robotics: A Survey of Recent Results," *Annals of Mathematics and Artificial Intelligence*, vol. 31, no. 1, pp. 113–126, 2001.
- [2] M. Dorigo, V. Maniezzo, and A. Coloni, "Ant System: Optimization by a Colony of Cooperating Agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996.
- [3] M. Dorigo, M. Birattari, and M. Brambilla, "Swarm Robotics," *Scholarpedia*, vol. 9, no. 1, p. 1463, 2014.
- [4] D. Karaboga and B. Basturk, "A Powerful and Efficient Algorithm for Numerical Function Optimization: Artificial Bee Colony (ABC) Algorithm," *Journal of Global Optimization*, vol. 39, no. 3, pp. 459–471, 2007.
- [5] J. Kennedy and R. Eberhart, "Particle Swarm Optimization," *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, pp. 1942–1948, 1995.

CONTINUATION PROJECTS (*complete if applicable, delete if not needed*)

Previous Year: *(Select the previous year's research)*

☐ 2024-2025 Season

☒ 2025-2026 Season

Previous Year's Project Title:

P.E.N.S.I.L. (Planetary Expeditionary Navigator and Settlement Infrastructure Launcher)

Research Goal - Current Year:

Design, implement, and benchmark a decentralized, momentum-driven vector coverage algorithm, Vel, utilizing Map-Aware Gradient Repulsion and Kinetic Vector Logic to optimize spatio-temporal coverage efficiency and victim discovery rates in simulated 2D and 3D disaster topologies compared to classical search algorithms, including Rigid CPP, PSO, ACO, and Levy Flight.

Research Goal – Previous Year:

To design a multi-rover swarm system that explores terrain significantly faster than conventional single rovers while reliably gathering data and achieving high map coverage efficiency across test trials.

Method/Procedure Summary – Current Year:

1. Hardware Baseline Validation (25% Scope): Assemble 3 ESP32 micro-rovers with ToF laser sensors and 18650 power supplies. Calibrate sensors, deploy ESP-NOW code, and conduct 50 trials in a bounded arena to measure peer-to-peer latency and agent velocity.
2. 3D Environment and Condition Setup (75% Scope): Build a Python simulation framework modeling discrete 3D voxel arrays (40x40x40) representing collapsed buildings, subterranean caves, rubble fields, and industrial shafts. Program physical constraints including RF signal blindspots, sensor noise, dynamic structural collapses, and variable obstacle densities (10% to 50%).
3. Core Protocol Implementation: Implement 3D Map-Aware Gradient Repulsion combining local attraction ($r = 2$) and global radial expansion. Integrate Kinetic Vector Logic continuous momentum updates ($\mu = 0.99$, $G = 1.5$, $\alpha = 0.12$) with sub-step interpolation ($dt = 0.1$) to prevent cell tunneling.
4. Baseline Algorithm Configuration: Adapt 3D Rigid Coverage Path Planning and 3D Stochastic Levy Flight as benchmark controls in identical voxel environments.
5. Monte Carlo Sweeps: Run automated simulation sweeps evaluating algorithms (Vel, 3D Rigid CPP, 3D Levy Flight), disaster topologies, swarm sizes ($N = 300, 500$), and environmental factors across fixed timesteps ($T = 50, 120$).

6. Data Logging: Log terminal volumetric coverage percentage, victim localization rates, structural collision counts, per-step processing latency, and continuous positional paths across all physical and simulated trials.

Method/Procedure Summary – Previous Year:

1. Hardware Setup and Component Testing: Assemble 3 2WD RC car chassis with Orange Pi 5B SBCs, Yahboom Silan LiDAR, MPU-6050 IMUs, Pi Cameras, and PCA9685 servo drivers. Flash Orange Pi OS onto microSD cards and test all components via SSH using Python smbus and pyserial libraries.
2. Circuit Fusing and Power Architecture: Wire the motors via ESCs and steering servos to the PCA9685 drivers. Power the microcontrollers and motor drivers using portable USB power banks, securing all components inside project boxes with solder and hot glue.
3. Reinforcement Learning Environment Design: Build a custom Gymnasium environment in Python for autonomous navigation. Define observation spaces (position, heading, 2D map) and action spaces (steering, throttle), structuring reward functions to optimize exploration, avoid obstacles, and prevent collisions.
4. Database Infrastructure: Install and configure MongoDB on a remote server to store real-time telemetry, trial performance, speed metrics, and matrix-based GPS updates transmitted over Wi-Fi.
5. Physical Trial Execution: Conduct 20 autonomous trial runs in a large, obstacle-rich 2D indoor test arena, randomizing agent starting positions and headings. Process continuous LiDAR scans to generate 2D maps until reaching the 95% coverage threshold or the 30-minute time limit.
6. Analytics and Performance Evaluation: Extract trial logs from MongoDB to calculate total mapping speed, GPS coverage matrices, and temporal efficiency (Y/X). Generate performance graphs in Python using Matplotlib and calculate mean metrics across all 20 runs.

Variables to be Tested – Current Year:

Independent Variables: Algorithmic steering protocols (Vel MAGR/KVL vs. 3D Rigid CPP vs. 3D Levy Flight), 3D disaster topology types, obstacle blockage density (10% to 50%), RF signal blindspots, and agent hardware failure rates.

Dependent Variables: 3D Volumetric Spatial Coverage Percentage, physical and simulated obstacle collision frequency, victim localization rate, and per-step algorithmic execution latency in milliseconds.

Variables Tested – Previous Year:

Independent Variables: System architecture (P.E.N.S.I.L. heterogeneous DRL swarm vs. single monolithic rover baseline) and Martian simulant terrain types (Vastitas Borealis vs. Noctis Labyrinthus).

Dependent Variables: Area Coverage Rate (ACR), Mean Traversal Efficiency (MTE in min/grid), linear velocity (m/s), and overall mission success rate (%).

Additional Changes/Differences – Current Year:

In Project Vel, we upgraded the software from 2D surface mapping to 3D volumetric disaster topologies (40x40x40 voxel grids). Instead of using heavy processing hardware and central DRL models like last year, we wrote a lightweight mathematical engine using Map-Aware Gradient Repulsion (MAGR) and Kinetic Vector Logic (KVL). Running on fused ESP32 micro-rovers over ESP-NOW wireless links, Vel navigates complex spaces without needing global coordinates or central servers.

Additional Changes/Differences – Previous Year:

Project P.E.N.S.I.L. established a solid, reliable foundation designed to get the job done effectively. It focused on speeding up 2D surface mapping across flat and rugged regolith testbeds using scout rovers. The setup relied on a standard, high-compute setup—using Orange Pi single-board computers, onboard LiDAR sensors, MongoDB cloud logging, and broad Deep Reinforcement Learning (DRL) policies to navigate terrain. While it proved that a multi-agent system could outperform a single rover, it relied on centralized computing, continuous Wi-Fi connectivity, and heavy sensor hardware.

Note: *(You will be asked to provide a copy of previous year's Research Plan & Abstract in PDF format)*